- Any application in OpenShift contains multiple gears (each gear has memory, bandwidth, CPU and disk allocated to it). We can either use single or multiple gears to create applications. Sometimes, one application is part of one gear and the other application is part of another gear, so to manage them we need an orchestrator. OpenShift broker manages orchestration and scheduling activities. In OpenShift v3 we leverage the advantage of Kubernetes, which is used along with an OpenShift broker to handle large scale cluster based applications.
- The OpenShift dashboard is user-friendly, and any kind of application development in Java, PHP, Python, Node.js or other supported languages is much easier and quicker than in other PaaS service providers.
- OpenShift has wide community support as Red Hat is actively participating in many open source projects, and OpenShift is the combined effort of all those communities.

Figure 1: OpenShift v3.0 stack

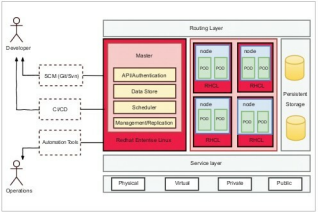

Figure 2: OpenShift v3.0 architecture

OpenShift v3.0 architecture

Figure 2 demonstrates the overall architecture of OpenShift v3.0.

As we have seen in Figure 1, OpenShift v3.0 follows the layered approach that leverages the benefits of Docker and Kubernetes. OpenShift v3.0 focuses more on how we can combine different services and compose an application easily. For example, in v3.0 we install Python, push code and then add MySQL.

However, OpenShift v2.0 provides configuration flexibility after composing all components. In v2.0, the application is used as a separate object, which is removed in v3.0 to provide flexibility in terms of combining various services— like different containers can reuse the same databases and expose them directly to the edge of the network.

OpenShift provides the functionalities listed below:

- Code management, build management and deployment
- Application management (elasticity, scalability and load balancing)
- Managing images in scale systems
- Team and user management for any organisation

The OpenShift architecture is made up of different small components, which run on top of the Kubernetes cluster where data about all the objects is stored in *etcd* (it's a master state—other components will look for this state and if any changes occur, then those components will change themselves into the desired state). It is also configured for high availability and deployed with 2n+1 peer services. REST APIs are used for communication between core objects. The controller reads those APIs and makes changes accordingly, to different objects. For example, when the request comes for build from the user side, a build object is created. The build controller then sees this object and runs a process to perform that build. Once the build is completed, the controller updates the status via the REST API to the build object, and the user can see that the build is over.

OpenShift follows the controller pattern, which provides flexibility and extensibility. We can define the behaviour of the controller whenever any object is created. Basically, it's business logic—it takes the controller and can also leverage the benefits of APIs to do any kind of automation and optimisation using scripting languages. We can also schedule cron jobs for any specific administrative tasks.

The controller should be in sync with the system and users, which happens due to even streams that push data from *etcd* to REST API and controller, so that the controller can quickly perform the required actions. In a failure scenario, the controller should resync all the data and resume the tasks accordingly.

OpenShift uses OAuth tokens and SSL certification for authorisation. A client typically makes API calls using a Web console or client program, and uses OAuth tokens for all the communications. Infrastructure (nodes) validation is done by client certificates generated by the systems that own the identities. Components inside containers use tokens to connect the APIs.

The OpenShift policy engine handles the authorisation part by creating different policies for a set of users and groups. Whenever any service account or user tries to perform an action, the engine checks for its identity before approving it.

Every container associates with the service account in the cluster. We can associate secrets (this is a service that handles all the passwords) with the service account, and automatically deliver these into the container. This provides the infrastructure the flexibility to manage secrets and perform different actions like pushing images, and also allows the application code to benefit from the secrets.